StockWise:

AI-Powered Decision System for Inventory Reordering and Waste Reduction

System Analysis Document (SAD)

1. Introduction

1.1. Purpose

This System Analysis Documentation (SAD) details the technical scope, architectural design decisions, data flows, functional/non-functional requirements, and integration of the GLM AI service layer for StockWise. It serves as the single source of truth for developers, QA testers, and stakeholders to understand how the system transforms raw inventory observations into actionable AI-driven recommendations while ensuring persistence, reliability, and graceful degradation.

Key elements covered:

- **Architecture**: Modular FastAPI backend with Supabase persistence, in-memory caching, and GLM as a dedicated service layer.
- **Data Flows**: Canonical input normalization → observation persistence → history collapse → analysis/snapshot → AI consumption.
- **Model Process**: End-to-end user journey from data ingestion to simulation, explanation, and chatbot chat.
- **Role of Reference**: This document enables modular development, testing traceability (linked to QATD), and future enhancements while preventing scope creep.

1.2. Background

Small cafe and kiosk operators face manual, error-prone inventory management: overstocking leads to waste (especially perishables), understocking causes stockouts, and decision-making relies on intuition rather than data. Existing manual processes (spreadsheets or basic POS systems) lack trend analysis, waste-risk scoring, or AI insights. StockWise is a greenfield MVP designed to solve this.

Architectural Innovations in this Version:

- Introduction of Supabase for user-scoped persistence (observations, analysis snapshots).
- Canonical input contract unifying CSV and manual entry.
- Dedicated GLM service with mock/fallback modes for offline development and resilience.
- In-memory analysis caching for low-latency performance.
- History-aware data collapse
- Real-time record CRUD with re-analysis.
- Reorder simulation with projected financial/risk impacts.
- Structured GLM explanations + grounded AI chatbot chat.
- Best-effort persistence with deterministic in-memory fallback.

1.3. Target Stakeholder

Stakeholders	**Roles**	**Expectations**
Cafe/Restaurant Owners (Primary Users)	Upload/manage inventory data; review KPIs/recommendations; simulate reorders; consult AI explanations/chatbot	Intuitive UI, accurate actionable insights (RESTOCK_NOW etc.), fast analysis (<2s), clear error handling, data privacy via Supabase ownership

Development Team	Implement backend services, frontend flows, Supabase integration, GLM adapter	Clean modular contracts (canonical schema), testable services, fallback mechanisms for AI
QA Team	Validate ingestion, analysis accuracy, simulation correctness, AI output quality	Defined input/output schemas, edge-case coverage (malformed CSV, hallucinations), regression on history collapse
Hackathon Judges/Evaluators	Assess technical feasibility, AI integration depth, MVP completeness	Demonstrable end-to-end flow with live GLM, robust architecture, clear documentation

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview

StockWise is a **full-stack client-server web application** (not mobile-native). The frontend (Next.js/TypeScript) interacts with a FastAPI backend via RESTful JSON APIs. All data is user-scoped in Supabase (PostgreSQL with Row Level Security). External dependency: GLM API (https://api.ilmu.ai/v1/chat/completions, model ilmu-glm-5.1). In-memory caching (keyed by analysis_id) ensures low-latency responses during the hackathon MVP.

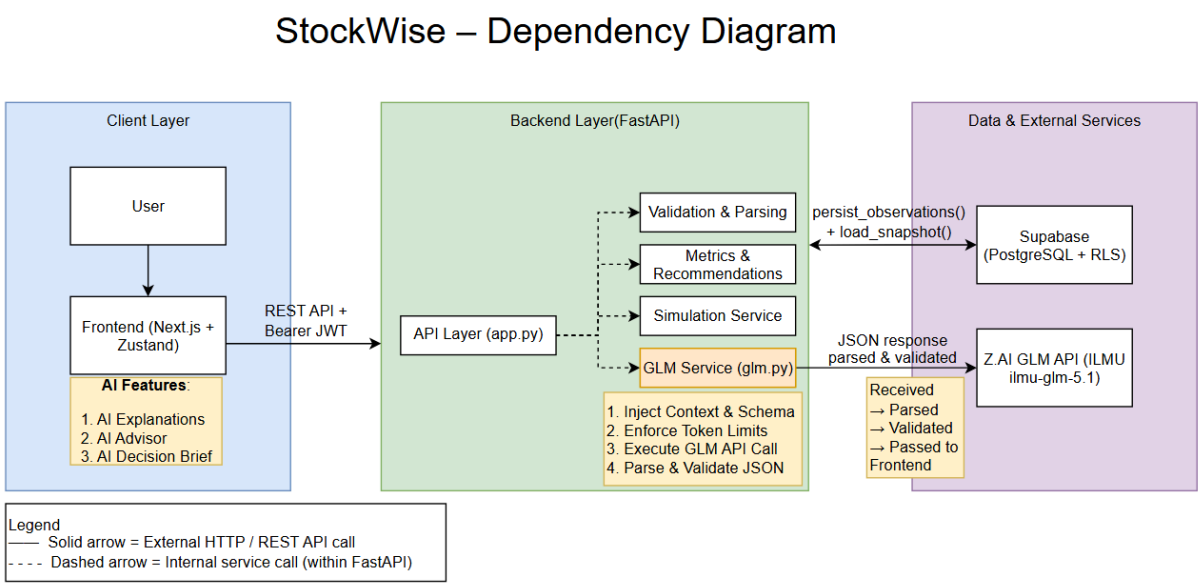
2.1.2 LLM as Service Layer

GLM is **not** a generic “AI Module” but a dedicated, pluggable service (src/stockwise_api/services/glm.py) with three modes:

- **live**: Calls the real ILMU endpoint.
- **mock**: Deterministic JSON responses for offline dev/testing.

- **fallback**: Pre-written templates triggered on invalid JSON, hallucinations, or timeouts.

2.1.3 Dependency Diagram



Explanation of Dependency Diagram:

a. How the prompts are being constructed and sent to the GLM API

The GLM Service (src/stockwise_api/services/glm.py) dynamically constructs prompts using carefully designed system and user message templates. It injects a strict JSON schema together with the current analysis snapshot, item metrics, recommendations, and simulation delta into the payload, then sends it via POST request to the ILMU GLM API (https://api.ilmu.ai/v1/chat/completions, model ilmu-glm-5.1).

b. What goes into the context window?

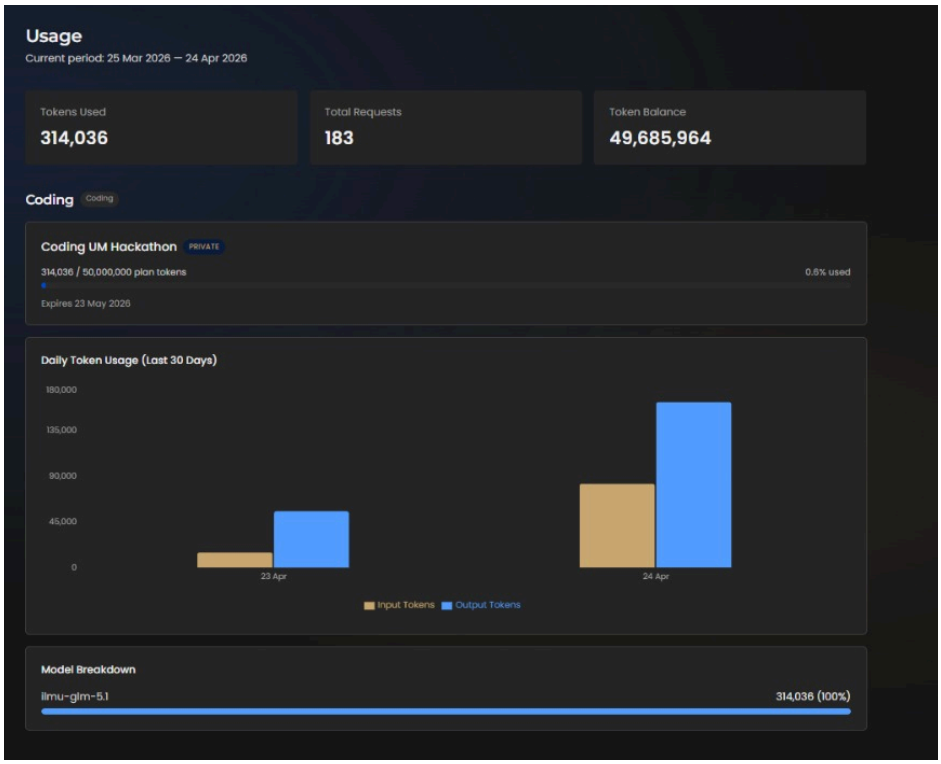
The context window contains the collapsed latest inventory observations (after history normalization), computed KPIs and metrics (e.g., days_of_cover, waste_risk_score, reorder_urgency_score), ranked recommendations, and any active simulation delta.

c. How does the system receive, parse responses and pass to the next system component?

The backend receives the JSON response from the GLM API, parses it, performs strict schema validation, and — on successful validation — immediately passes the structured output to the frontend (explanation drawer, AI chat component, or AI Decision Brief Card). Invalid responses trigger retry logic or deterministic fallback templates.

d. Where the limitation of tokens are enforced or input are chunked before it actually reaches GLM?

Token limitation and input chunking are enforced **inside the GLM Service (glm.py)** before the request is sent to the external API. The service uses intelligent context compaction functions (_compact_model_context() and _compact_chat_context()) to strip unnecessary fields, prioritize the most recent critical data points, and truncate older historical observations when needed, while maintaining a safety margin. As of 24 April 2026, the project has consumed **314,036 tokens** across **183 requests** on ilmu-glm-5.1 (visible in the ILMU Console).



e. Major API calls between the system components

- Frontend (Next.js + Zustand + AI Decision Brief Card) → Backend Layer: POST /api/v1/analyses, POST /api/v1/analyses/{id}/explanation, POST /api/v1/analyses/{id}/ai-chat, and GET /api/v1/analyses/{id}/decision-brief (async / parallel) (REST API + Bearer JWT)
- API Layer (app.py) → Internal Services (dashed arrows): Validation & Parsing, Metrics & Recommendations, Simulation Service, GLM Service
- Services → Supabase: persist_observations() and load_snapshot()

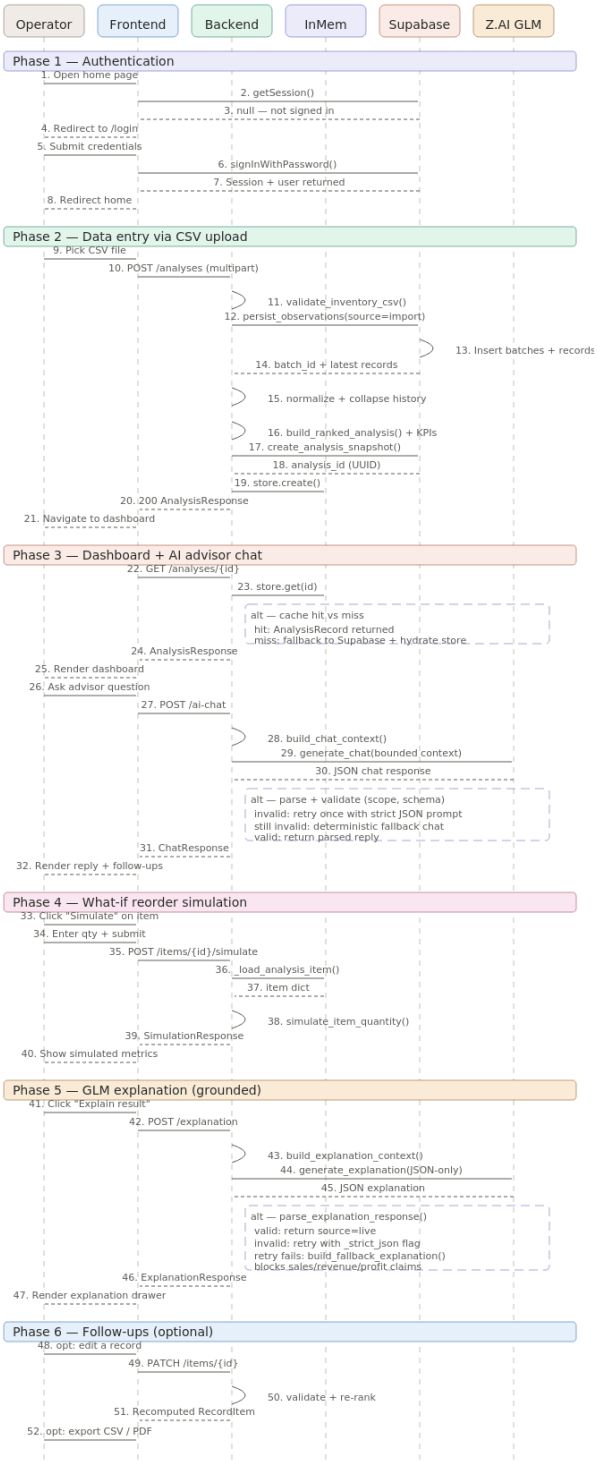
f. How frontend, backend, services, database and external API interact with each other?

The Next.js Frontend (including the new AI Decision Brief Card) makes authenticated REST API calls using Bearer JWT tokens (issued by Supabase Auth) to the Backend Layer (FastAPI). The AI Decision Brief is requested asynchronously in parallel with the main dashboard load so KPI cards and the analysis table render immediately without blocking.

g. Distributed components

The system is designed with clearly separated distributed components: Client Layer (Next.js + Zustand + AI Decision Brief Card), Backend Layer (FastAPI), Data Layer (Supabase cloud PostgreSQL with RLS), and External AI Service (ILMU GLM API ilmu-glm-5.1).

2.1.4 Sequence Diagram



a) User Interaction Flow / Customer Journey with Stakeholders

Flow Name: CSV Upload to AI-Powered Reorder Decision Flow

Stakeholders involved:

- Cafe Operator (End User) — a small cafe or kiosk owner-operator who is personally involved in purchasing and inventory control.
- Frontend (Next.js Web App) — the user-facing interface where the operator uploads data, views the dashboard, interacts with the AI advisor, runs simulations, and reads explanations.
- Backend (FastAPI Server) — the decision intelligence engine that validates data, computes derived metrics, runs ranking logic, and orchestrates AI calls.
- InMemoryAnalysisStore (InMem) — a server-side RAM cache that holds the active analysis result keyed by `analysis_id` so repeated dashboard reads do not require recomputation or a database round-trip.
- Supabase — the authentication provider and persistent datastore that owns user sessions, historical inventory records, import batches, and point-in-time analysis snapshots.
- Z.AI GLM — the large language model provider that transforms structured inventory metrics into operator-friendly explanations and grounded chat answers.

Scenario: A cafe operator logs into StockWise, uploads a 100-day inventory CSV, reviews the Decision Dashboard, asks the AI advisor what to buy today, simulates a reorder quantity for a specific item, and reads the GLM-generated explanation of the simulation result.

b) Step-by-Step Interactions

The journey is organised into six phases. Each step shows the participant initiating the action, the participant receiving it, and what happens between them.

Phase 1 — Authentication

1. The operator opens the home page in the browser.
2. The frontend calls `getSession()` against Supabase Auth to check for an existing session.
3. Supabase returns `null` because the operator has not yet signed in.

4. The frontend redirects the user to the `/login` page.
5. The operator submits their email and password.
6. The frontend calls `signInWithPassword()` on Supabase Auth.
7. Supabase validates the credentials and returns a session token plus the user object.
8. The frontend redirects the operator back to the home page, now authenticated.

Phase 2 — Data Entry via CSV Upload

9. The operator picks the inventory CSV file from their device.
10. The frontend sends `POST /api/v1/analyses` to the backend as a multipart form request.
11. The backend runs `validate_inventory_csv()` to check required columns, data types, and row integrity. Invalid files are rejected early with user-friendly feedback.
12. The backend calls `persist_observations(source=import)` on Supabase to save every valid row as historical inventory data.
13. Supabase inserts rows into `import_batches`, `suppliers`, `items`, and `inventory_records` under the authenticated user's ownership.
14. Supabase returns the new `batch_id` and the latest record per unique item.
15. The backend runs `normalize_item_history()` to collapse the 1,000-row history into one latest state per item grouped by stable item identity.
16. The backend runs `build_ranked_analysis()` to compute derived metrics (`days_of_cover`, `inventory_value`, `estimated_waste_cost`, `lead_time_demand`, `reorder_urgency_score`, `waste_risk_score`) and assigns each item a recommended action. It also builds the KPI summary.
17. The backend calls `create_analysis_snapshot()` on Supabase to persist the analysis run and its ranked item results as a point-in-time snapshot.
18. Supabase returns a freshly generated `analysis_id` (UUID).
19. The backend stores the analysis in InMem via `store.create()` for fast retrieval on subsequent dashboard reads.
20. The backend returns `200 AnalysisResponse` containing ranked items, KPIs, and dataset summary to the frontend.
21. The frontend navigates the operator to `/dashboard/{analysis_id}`.

Phase 3 — Dashboard Rendering and AI advisor Chat

22. The frontend sends `GET /api/v1/analyses/{analysis_id}` to load the dashboard data.
23. The backend calls `store.get(id)` on InMem to retrieve the cached analysis.
24. Alternative path (cache hit vs miss):

- *Cache hit:* InMem returns the `AnalysisRecord` instantly.
- *Cache miss:* The backend falls back to the Supabase snapshot, rehydrates InMem, and proceeds with the recovered data.

25. The backend returns the `AnalysisResponse` to the frontend.
26. The frontend renders the KPI cards, priority action board, and filter controls.
27. The operator types a question in the AI advisory panel (for example, *"What should I buy today?"*).
28. The frontend sends `POST /api/v1/analyses/{id}/ai-chat` with the message and recent conversation context.
29. The backend runs `build_chat_context()` to assemble a bounded payload containing only the current analysis items and KPI summary, never the raw 1,000 rows.
30. The backend calls `generate_chat()` on Z.AI GLM with the bounded structured prompt.
31. GLM returns a JSON chat response.
32. **Alternative path (parse and validate):**

- *Valid:* The backend parses the JSON and returns the structured reply.
- *Invalid:* The backend retries once with a stricter JSON-only prompt.
- *Still invalid or unavailable:* The backend falls back to a deterministic chat response so the user flow never breaks.

33. The backend returns `ChatResponse` containing the answer, supporting points, related items, and suggested follow-ups.
34. The frontend renders the AI advisor reply along with clickable follow-up chips.

Phase 4 — What-If Reorder Simulation

35. The operator clicks the Simulate button next to an item in the action board.
36. The operator enters a proposed reorder quantity and confirms.

37. The frontend sends `POST /api/v1/analyses/{id}/items/{item_id}/simulate` with the proposed quantity.
38. The backend calls `_load_analysis_item()` on InMem to retrieve the current state of that item.
39. InMem returns the item dictionary.
40. The backend runs `simulate_item_quantity()` to compute `simulated_coverage_days`, `simulated_cash_outlay`, updated urgency and waste scores, and a `simulated_risk_change` label.
41. The backend returns the `SimulationResponse` to the frontend.
42. The frontend displays the simulated metrics next to the current metrics so the operator can compare the trade-off.

Phase 5 — GLM Explanation (Grounded)

43. The operator clicks the Explain This Result button on the simulation screen.
44. The frontend sends `POST /api/v1/analyses/{id}/items/{item_id}/explanation` with the simulated order quantity.
45. The backend runs `build_explanation_context()` to assemble the structured payload (item identity, current metrics, derived metrics, recent context, and simulation context).
46. The backend calls `generate_explanation()` on Z.AI GLM with the instruction to return JSON only.
47. GLM returns a JSON explanation.
48. Alternative path (parse, validate, retry, fallback):

- *Valid:* The backend returns the explanation with `source=live`.
- *Invalid:* The backend retries once with the `_strict_json` flag.
- *Retry still fails:* The backend calls `build_fallback_explanation()` to return a deterministic template, and the response carries `source=fallback`.
- *Safety check:* The parser blocks any unsupported sales, revenue, or profit claims the model might hallucinate.

49. The backend returns the validated `ExplanationResponse` to the frontend.
50. The frontend renders the Explanation Drawer containing the recommended action, trade-off summary, next step, and confidence note.

Phase 6 — Follow-Ups (Optional)

51. If the operator spots an error in a record, they edit it on the records page.
52. The frontend sends `PATCH /api/v1/analyses/{id}/items/{item_id}` with the updated field values.
53. The backend validates the change, updates InMem, and re-runs the ranking logic so recommendations reflect the corrected data.
54. The backend returns the recomputed `RecordItem` to the frontend.
55. The operator can also export the analysis as a CSV or PDF from the export page. This runs client-side via a browser Blob download.

c) Demonstration of Each Key System Feature from Start to Finish

This section maps each StockWise feature in the PRD to the steps above.

Schema Validation and Data Ingestion (FR1, FR2) — Steps 9–11. The operator selects a CSV, the backend validates the 14-field contract, and invalid files are rejected before any analysis runs.

Persistent Observation History (FR17, FR18, FR23) — Steps 12–15. Every validated row is saved to Supabase, then 1,000 daily records collapse into one latest state per item.

Derived Metric Computation and Ranking (FR3–FR7) — Step 16. The backend computes all derived metrics and assigns each item one of four actions: RESTOCK_NOW, BUY_LESS, DELAY_PURCHASE, or MONITOR_CLOSELY.

Analysis Snapshot Persistence (FR24, FR25) — Steps 17 and 24. Snapshots are written to Supabase and recovered on cache miss, so a server restart never loses the operator's analysis.

Decision Dashboard (FR8) — Steps 22–26. KPI cards, priority action board, and filters let the operator identify priority items in under two minutes.

AI advisor Chat (FR26–FR28) — Steps 27–34. Bounded context prevents oversized input, and the validate-retry-fallback branch demonstrates graceful degradation.

What-If Reorder Simulator (FR9) — Steps 35–42. The operator compares aggressive vs conservative orders before deciding.

GLM Explanation Layer (FR10–FR13) — Steps 43–50. The three-tier resilience (live, retry, fallback) is the clearest proof that GLM is essential, not decorative.

Record Review, Edit, and Recompute (FR20–FR22) — Steps 51–54. Operators correct source data and immediately see updated recommendations.

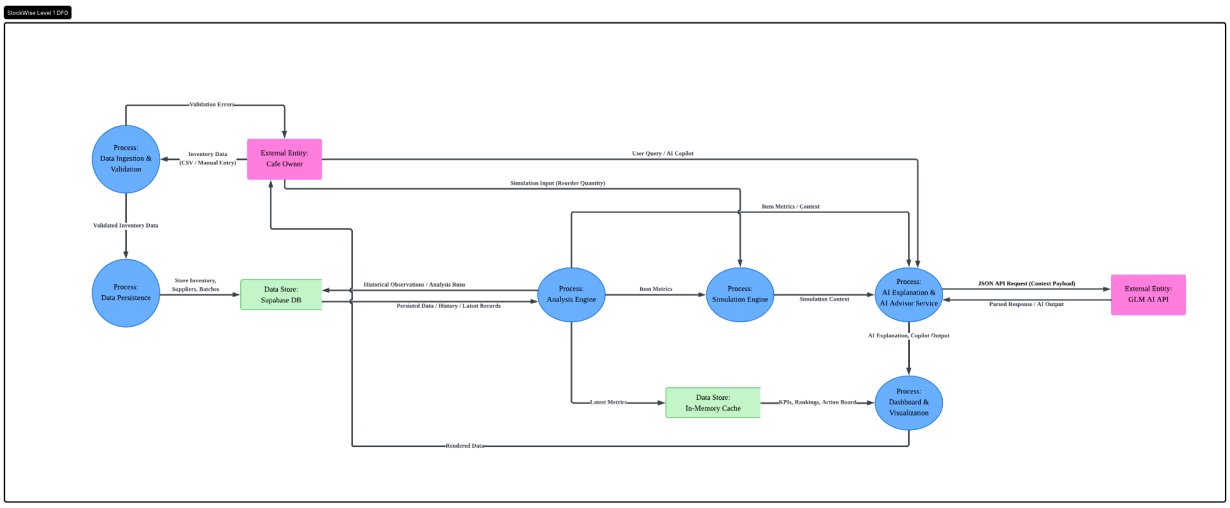
Resilient GLM Integration Pattern — Steps 32 and 48. Both GLM touchpoints follow the same live → retry → fallback pattern, keeping the product demo-safe.

2.2. Technological Stack

- **Frontend**: Next.js 14 (App Router) + TypeScript + Tailwind CSS + Zustand (state management).
- **Backend**: FastAPI (Python) + Uvicorn + Pydantic schemas.
- **Database**: Supabase (PostgreSQL) with auth, migrations, and RLS.
- **AI Layer**: GLM (ILMU API) with mock/fallback.
- **Other**: In-memory cache, CSV parsing, PyTest (existing tests/).

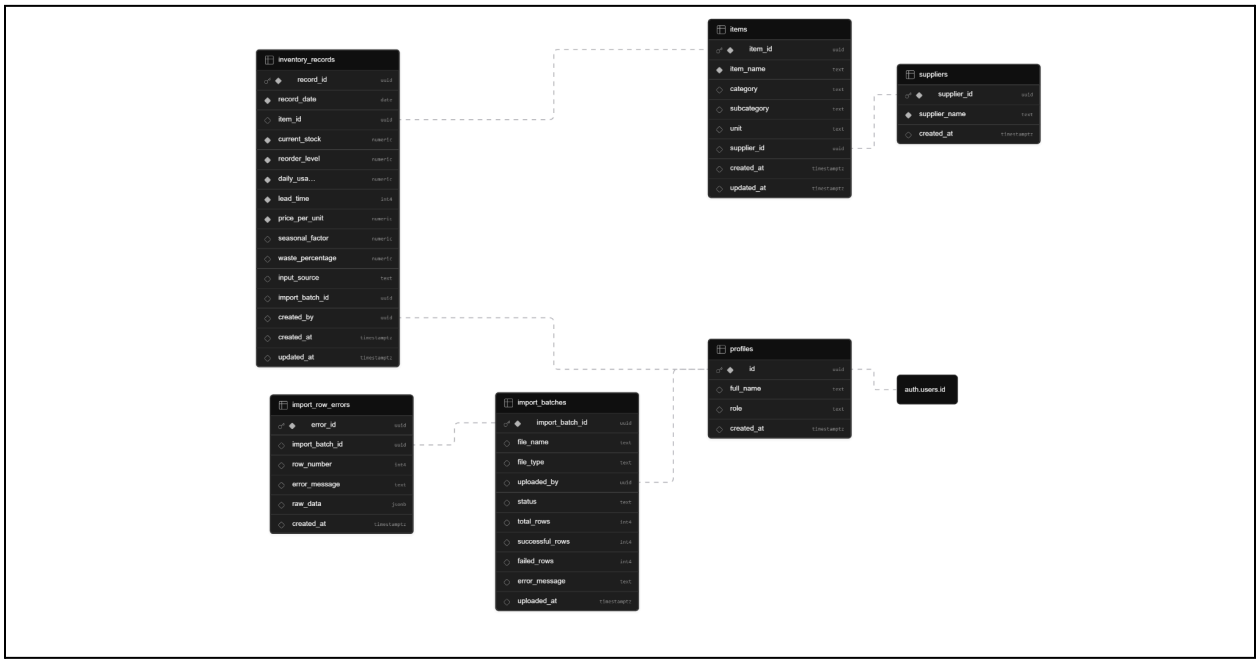
2.3. Key Data Flows

2.3.1. Data Flow Diagram (DFD)



Based on the provided Data Flow Diagram, the system operates through a sequential and interactive data pipeline starting with the **Cafe Owner**, who submits raw inventory data that is first validated and then securely stored in the **Supabase DB**. The **Analysis Engine** retrieves this historical data to compute current item metrics, caching the baseline KPIs and rankings in an **In-Memory Cache** for rapid dashboard rendering. When the user actively engages with the system—either by inputting a reorder quantity for simulation or asking a direct question—the **Simulation Engine** and **Analysis Engine** feed the localized context and item metrics into the **AI Explanation & AI advisor Service**. This service bundles the contextual data into a JSON API request sent to the external **GLM AI API**, which returns parsed AI insights. Finally, the **Dashboard & Visualization** process merges these live AI explanations with the cached metrics to deliver comprehensive, rendered decision data back to the **Cafe Owner.**

2.3.2. Normalized Database Schema



3. Functional Requirements & Scope

3.1. Minimum Viable Product

#	Features	Description
1	Inventory Ingestion	CSV upload (header mapping + validation) **or** manual entry; both converge to canonical contract and persist as observations.
2	Analysis & Recommendations	Collapse history → compute KPIs (days_of_cover, waste_risk_score, etc.) → ranked items with actions (RESTOCK_NOW etc.).
3	AI Decision Brief	Dashboard-level AI Decision Brief generated asynchronously via GLM. Loads in parallel with the main dashboard; delivers prioritized actions (Buy Today / Buy Less / Delay), estimated impact, trade-offs, and recommended order with graceful fallback.
4	Reorder Simulation	What-if reorder quantity → projected cash outlay, coverage, risk delta.
5	AI Insights (Explanation + chatbot)	Structured GLM explanations per item + grounded chat scoped to current analysis/simulation.

3.2. Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Scalability	Handle 1000+ row CSVs and concurrent analyses during peak use	Batch processing + in-memory cache; Supabase scaling

Reliability	100% deterministic core analysis; no data loss on persistence failure	Best-effort Supabase writes + in-memory fallback; JSON validation + GLM fallback
Maintainability	Modular services with clear contracts	Separate files for validation/metrics/recommendations/glm; existing tests/
Token Latency	GLM responses <3s (p95) under normal load	Timeout handling, context chunking, mock mode
Cost Efficiency	Minimize token usage per session	Fixed-length prompts, caching of repeated analyses, fallback templates
Security	User-scoped data only	Supabase RLS + JWT auth on all endpoints

3.3. Out of Scope / Future Enhancements

- Multi-user collaboration / sharing of analyses.
- Advanced reporting/export beyond current dashboard/history.
- Real-time notifications or mobile push.
- Integration with external POS/ERP systems.
- Production monitoring dashboards (Prometheus/Grafana).

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation

4.1.1. Grayscale Rollout & A/B Testing

For post-hackathon deployment: New features (e.g., enhanced simulation UI) released to 10% of authenticated users via feature flags in Supabase; monitor error rates and AI pass rate for 24h before 100% rollout. A/B test deterministic vs. live GLM fallback impact on user engagement.

4.1.2. Emergency Rollback (ER) & Golden Release

GitHub Actions (or Vercel) maintains “golden” main-branch deployment. On critical issues (e.g., >5% AI hallucination rate or Supabase outage), automated rollback to previous stable tag + in-memory-only mode.

4.1.3. Priority Matrix

- **P0 Critical** (e.g., data corruption on ingest, 500 on analysis): Immediate rollback + hotfix within 1 hour.
- **P1 High** (e.g., AI fallback not triggering): Block merge; must mitigate before production.
- **P2 Medium** (e.g., UI polish): Defer to next sprint.

4.2. Monitoring

- Backend: FastAPI logging + Supabase query logs.
- AI: Log source (live/mock/fallback) + confidence_note; alert on >10% fallback rate.
- Frontend: Console errors + Zustand state inspection.
- Health checks: /health endpoint + Supabase connection test on startup.

4.3. Assumptions

- Users have a stable internet (for Supabase + GLM calls).
- Owners provide reasonable estimates for required fields (price, seasonality, waste signals).

- The Hackathon environment has a valid GLM API key and Supabase project.
- No extreme concurrency beyond MVP scale.

4.4. External Dependencies

Tools	**Purpose**	**Risks & Mitigation**
Supabase (PostgreSQL + Auth)	Persistence, user scoping, RLS	Outage → in-memory fallback; rate limits → batching
GLM API (ILMU)	Core reasoning for explanations & chatbot	Rate limits/hallucinations → mock + deterministic fallback + JSON validation + retry
GitHub	Version control & CI	None (local dev possible)

5. Project Management & Team Contributions

5.1. Project Timeline

- Day 1: Planning + canonical schema + basic ingestion (CSV/manual).
- Day 2-3: Analysis engine, persistence, records CRUD.
- Day 3-4: Simulation + GLM integration (mock → live).
- Day 5-6: Frontend unification + AI chatbot + testing.
- Day 7-8: Documentation (PRD + SAD + QATD) + demo video.

5.2. Team Members Role

- **Project Manager & QA (Ku Kian Xiang):** SAD/QATD ownership, sprint planning, and AI accuracy testing.
- **Frontend Lead (Chuah Teik Shun):** Next.js pages, Zustand state management, and dashboard UI.
- **Backend Lead (Lo Jia Keng):** FastAPI services, scoring algorithms, and API orchestration.
- **Data & DevOps (Thong Shu Heng):** Supabase migrations, RLS security, and deployment pipeline.
- **AI Strategy (Lim Wey Cheng):** Prompt engineering, GLM fallback logic, and context compaction.

5.3. Recommendations

- Add Redis (or Supabase Edge Functions cache) for production scaling of analysis cache.
- Implement rate limiting on GLM calls.
- Expand test coverage in /tests/ for all edge-case normalization paths.
- Consider Vercel + Supabase for zero-ops deployment.